

Problem Set 6: Solution

1. Conv1d and the sliding dot product

Given the DSP convolution sum $y_{DSP}[n] = \sum_k h[k]x[n-k]$ and PyTorch's cross-correlation convention $y_{torch}[n] = \sum_k w[k]x[n+k]$:

(a) Derive the relationship $w[k] = h[M-k]$ (where h has $M+1$ taps, indices $0, \dots, M$) that makes `F.conv1d` reproduce the textbook causal FIR convolution, and state the required left-padding amount.

$$y_{DSP}[n] = \sum_{l=0}^M h[l] \cdot x[n-l] = \sum_{k=M}^0 h[M-k] \cdot x[n+k-M]$$

$M-k \leftarrow l$

$$\text{set } w[k] = h[M-k] \Rightarrow = \sum_{k=0}^M w[k] \cdot x[n+k-M]$$

$$\text{compare to } y_{torch}[n] \rightarrow = y_{torch}[n-M]$$

shifted right by M

$\Rightarrow$ need to pad $x[n]$ with M zeros on the left for `conv1d`

For comparison with 'full' `numpy.convolve()` result padding of $x[n]$ with M zeros is also needed on the right for `conv1d`.

(b) Implement both, the NumPy convolution `np.convolve` and the equivalent `F.conv1d` computation for a Hamming-windowed FIR lowpass filter with $\omega_c = \pi/3$ and length $N = 51$ on a chirp signal (`chirp_001.wav`). Report the maximum absolute difference between the two outputs (you should get something on the order of 10^{-6} to 10^{-7} in float32). Show evidence that your filter works as desired (e.g., by recording the filtered chirp signal as .wav-file and using a program like Audacity to generate a spectrogram). Show all your Python and PyTorch code in your solution.

Python code, part 1

```
import numpy as np
import scipy.signal as ss
import matplotlib.pyplot as plt
import IPython.display as ipd
from scipy.io import wavfile

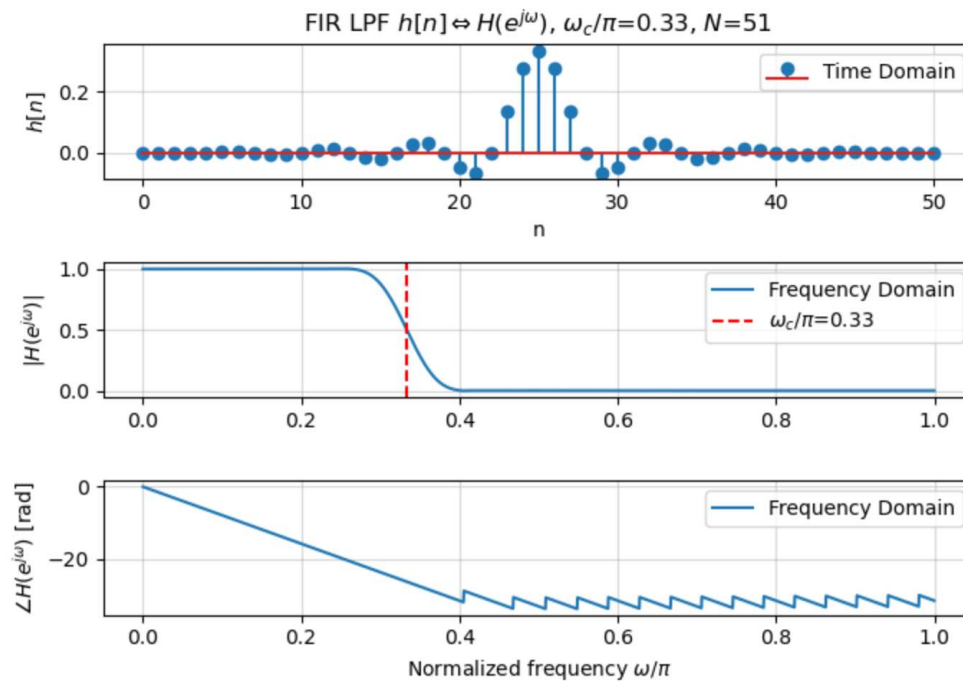
# Read .wav file
def wav16read(f_name):
    fs, xn16 = wavfile.read(f_name) # read sampling rate Fs and data xn from .wav file
    return fs, xn16.astype(np.float32)/2**15 # convert to 32-bit floating point in range -1.0 to +1.0

# Write .wav file
def wav16write(f_name, fs, yn):
    yn16 = np.array(2**15*yn, np.int16)
    wavfile.write(f_name, fs, yn16)

# Load chirp signal
fs, xn = wav16read('audio/chirp_001.wav')

# FIR Lowpass Filter
Ntaps = 51
M = Ntaps - 1 # filter order
wc = np.pi/3 # cutoff frequency in radians
hlp = ss.firwin(numtaps=Ntaps, cutoff=wc/np.pi, window='hamming') # impulse response
```

Lowpass filter, time and frequency domain response



Python code, part 2

```
# DSP convolution to low-pass filter the chirp signal
yDSP = np.convolve(xn, hlp, mode='full')
wav16write('audio/chirp_001_yDSP.wav', fs, yDSP)
print(yDSP.size)

16050

# Importing PyTorch modules
import torch
import torch.nn.functional as F

# Flipped filter coefficients w_torch and padded xn signal
# w[k] = hlp[M-k]:
w_torch = torch.tensor(np.array(hlp, np.float32)).view(1, 1, -1).flip(dims=[2])
# x_padded[n] = x[n-M], M is filter order of hlp[n] Lowpass filter
xn_padded = F.pad(torch.tensor(xn).view(1, 1, -1), pad=(M, M))

# Execute (full) convolution in PyTorch
y_torch = F.conv1d(xn_padded, w_torch).squeeze()
wav16write('audio/chirp_001_ytorch.wav', fs, y_torch)
print(y_torch.shape)

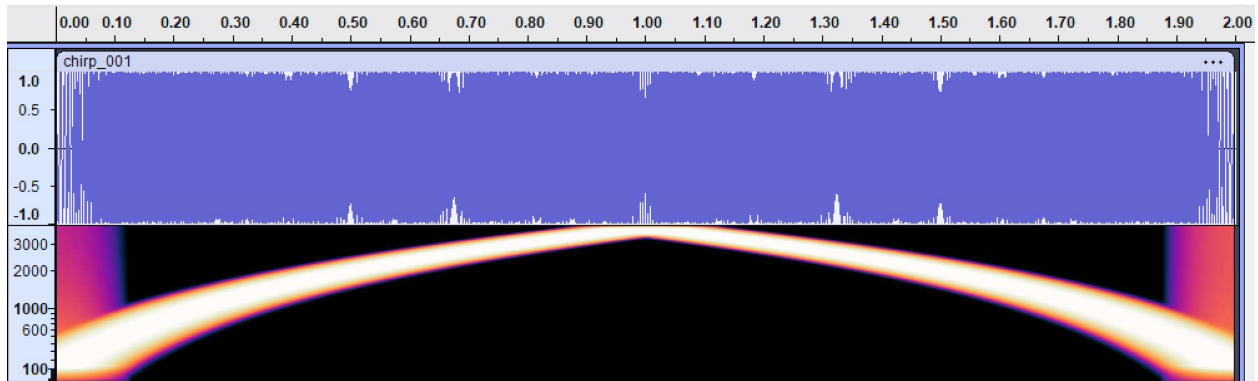
torch.Size([16050])

# Compare DSP convolution and conv1d result
print(f'max error: {np.max(np.abs(yDSP - y_torch.tolist()))}')

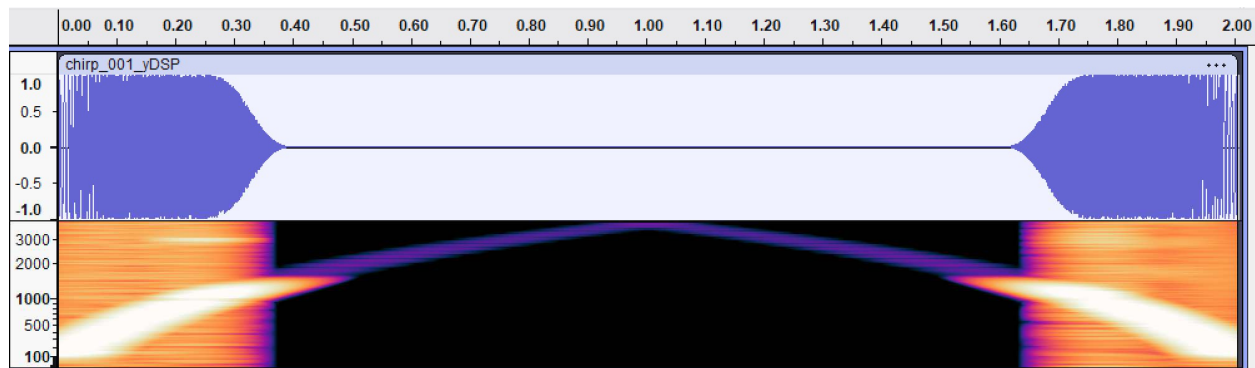
max error: 3.590034161060629e-07
```

Note that the maximum error between the `numpy.convolve()` and the `torch.nn.functional.conv1d()` is only about 3.59×10^{-7} .

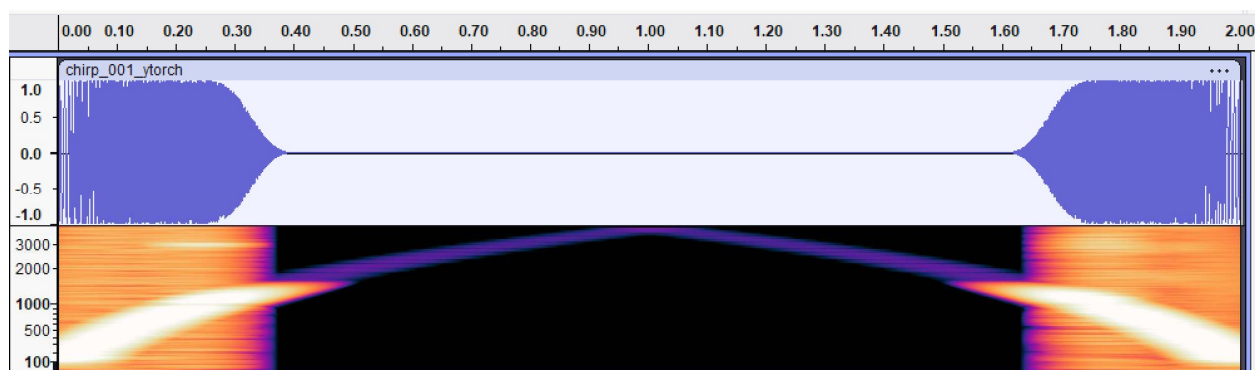
Time waveforms and spectrograms from Audacity, original Chirp signal:



After DSP low-pass filtering:



After conv1d low-pass filtering:



(c) A colleague says “PyTorch’s `conv1d` does convolution backwards, so it’s a bug.” Give a one-paragraph rebuttal, using the ML viewpoint discussed in lecture.

Rebuttal: It's not a bug, just a different (and for machine learning purposes, more natural) convention. Mathematically, convolution and cross-correlation differ only by flipping the kernel, $w[k] = h[M - k]$, exactly as derived in part (a), so no information or capability is lost, and the two are trivially interconvertible. In classical DSP the kernel $h[k]$ is a *fixed, physically meaningful* impulse response, derived from a difference equation or an analog prototype, so the flip in the definition matters and must be tracked carefully. In deep learning, however, the kernel $w[k]$ is a set of *learned parameters* with no prior physical meaning: gradient descent will happily learn whichever set of numbers minimizes the loss, whether the operation is called “convolution” or “correlation.” Since training and inference both consistently use the same cross-correlation definition, and autograd computes the correct corresponding gradients automatically, there is no mismatch anywhere in the pipeline. The “backwards” framing only becomes an issue when reusing a fixed, externally designed filter, like the FIR filter in part (b), inside a `conv1d` call, and even then, as shown above, it is a one-line fix: flip the kernel and left/right-pad by M , not a bug.

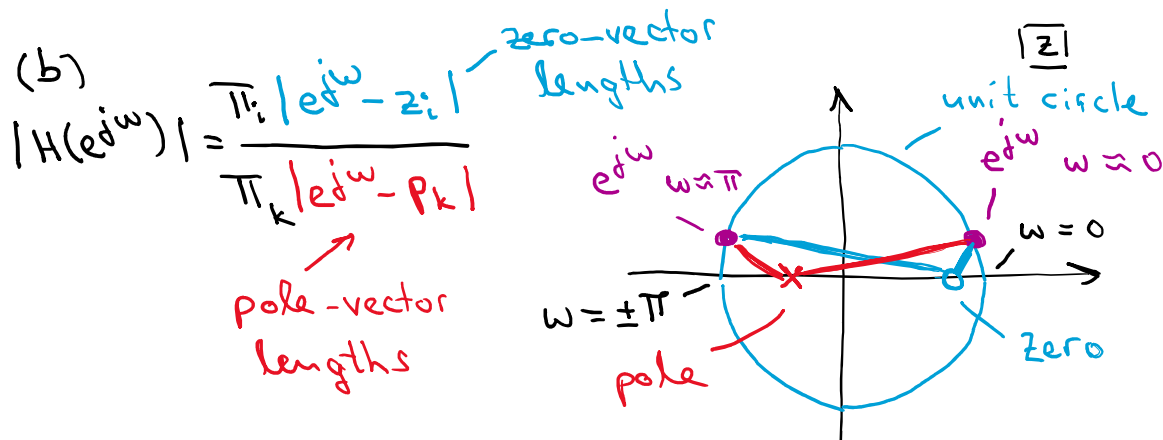
2. Pole-zero geometry

For

$$H(z) = \frac{1 - 0.8z^{-1}}{1 + 0.5z^{-1}}$$

- Identify the zero and the pole.
- Explain geometrically why $|H(e^{j\omega})|$ is the product of zero-vector lengths divided by the product of pole-vector lengths.
- Predict qualitatively whether the magnitude is larger near $\omega = 0$ or $\omega = \pi$.

(a) zero at $z = 0.8$
pole at $z = -0.5$



Each factor $|e^{j\omega} - z_i|$ is the distance between $e^{j\omega}$ and the zero at z_i . $|H(e^{j\omega})|$ is proportional to $|e^{j\omega} - z_i|$ since the factor appears in the numerator. Each factor $|e^{j\omega} - p_k|$ is the distance between $e^{j\omega}$ and the pole at p_k . $|H(e^{j\omega})|$ is inversely proportional to $|e^{j\omega} - p_k|$ since the factor appears in the denominator.

(c) At $w \approx 0$ $|e^{j\omega} - z_0| < |e^{j\omega} - p_0| \Rightarrow$ small $|H(e^{j\omega})|$
At $w \approx \pi$ $|e^{j\omega} - z_0| > |e^{j\omega} - p_0| \Rightarrow$ large $|H(e^{j\omega})|$
 $\Rightarrow$ Highpass leaning response.

3. Phase delay and group delay

Let $H(e^{j\omega}) = |H(e^{j\omega})|e^{j\phi(\omega)}$ with $\phi(\omega) = -3\omega + 0.4\sin(\omega)$.

(a) Compute the phase delay

$$\tau_p(\omega) = -\frac{\phi(\omega)}{\omega}$$

for $\omega \neq 0$.

(b) Compute the group delay

$$\tau_g(\omega) = -\frac{d\phi(\omega)}{d\omega}.$$

(c) Explain why the system is not linear phase.

$$(a) \quad \tau_p(\omega) = \frac{-\phi(\omega)}{\omega} = 3 - 0.4 \frac{\sin(\omega)}{\omega} \quad \omega \neq 0$$

$$(b) \quad \tau_g(\omega) = -\frac{d\phi(\omega)}{d\omega} = 3 - 0.4 \cos(\omega)$$

(c) Linear phase requires $\phi(\omega) = -\alpha\omega$ for some constant α , thus resulting in

$\tau_g(\omega) = -\frac{d\phi(\omega)}{d\omega} = \alpha$. As seen in (b), there is an additional function of ω ($\cos(\omega)$) in $\tau_g(\omega)$ for this system that shows that the phase is not linear.

4. First order allpass

For real $|a| < 1$,

$$H_{ap}(z) = \frac{z^{-1} - a}{1 - az^{-1}}$$

Show that

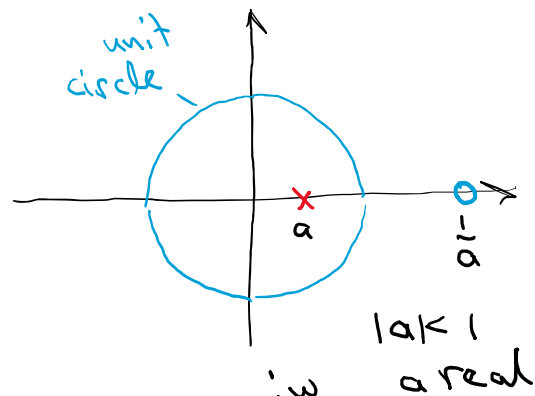
$$|H_{ap}(e^{j\omega})| = 1$$

for all ω . Locate the pole and zero and explain the reciprocal relationship.

Zero: $z^{-1} - a = 0 \Rightarrow z^{-1} = a$ or $z = \frac{1}{a}$ zero

Pole: $1 - az^{-1} = 0 \Rightarrow az^{-1} = 1$ or $z = a$ pole

The pole and zero are in reciprocal locations, i.e., at a and $\frac{1}{a}$



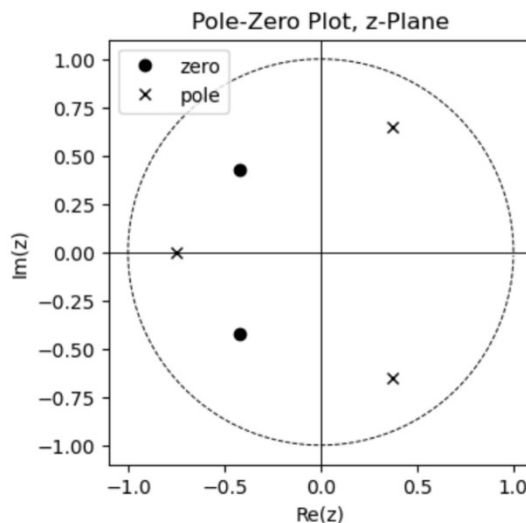
$$H_{ap}(e^{j\omega}) = \frac{e^{-j\omega} - a}{1 - ae^{-j\omega}} = e^{-j\omega} \frac{1 - ae^{j\omega}}{1 - ae^{-j\omega}}$$

Taking magnitude and noticing that numerator and denominator of rational expression are complex conjugates we obtain

$$|H_{ap}(e^{j\omega})| = \underbrace{|e^{-j\omega}|}_{=1} \frac{|1 - ae^{j\omega}|}{\underbrace{|1 - ae^{-j\omega}|}_{=1}} = 1$$

5. Inverse system properties (ECEN 5632 Problem)

If the system function $H(z)$ of an LTI system has a pole-zero plot as shown on the right and the system is causal, can the inverse system $H_i(z)$, where $H(z)H_i(z) = 1$, be both causal and stable? Clearly justify your answer.



Let $z_0 = r_0 e^{\pm j\theta_0}$ be the zeros of $H(z)$ and

let $p_1 = a_1$ and $p_2 = r_2 e^{\pm j\theta_2}$ be the poles of $H(z)$

$$\begin{aligned} \text{Then } H(z) &= \frac{(z - r_0 e^{j\theta_0})(z - r_0 e^{-j\theta_0})}{(z - a_1)(z - r_2 e^{j\theta_2})(z - r_2 e^{-j\theta_2})} \\ &= \frac{z^2 - 2r_0 \cos(\theta_0)z + r_0^2}{z^3 - [2r_2 \cos(\theta_2) + a_1]z^2 + [r_2^2 + 2a_1 r_2 \cos(\theta_2)]z - a_1 r_2^2} \end{aligned}$$

This has all the poles and zeros shown in the pole-zero plot, but because the denominator has degree 3 in z and the numerator only has degree 2, it also has a zero at infinity (as $z \rightarrow \infty$).

Thus, the inverse system would have a pole at $z = \infty$ (in addition to poles at $r_0 e^{\pm j\theta_0}$).

Therefore the inverse system $H_i(z)$ cannot be both causal and stable.